

Problem 1 - Simulation of Sources from Monopoles

The Matlab code for this problem is listed in Appendix 1.

Problem 1a

Figure 1 shows the spherical spreading from a monopole. The frequency is 1 kHz. The sound pressure level has been calculated at two distances: 1 m and 4 m.

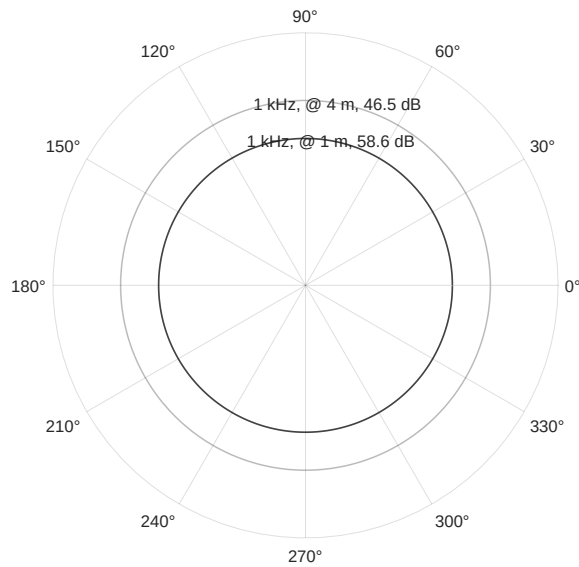


Figure 1: Monopole spherical spreading for 1 kHz. The radius of the polar plot is the sound pressure level.

Figure 2 shows the sound pressure level as a function of distance for a 1 kHz monopole source.

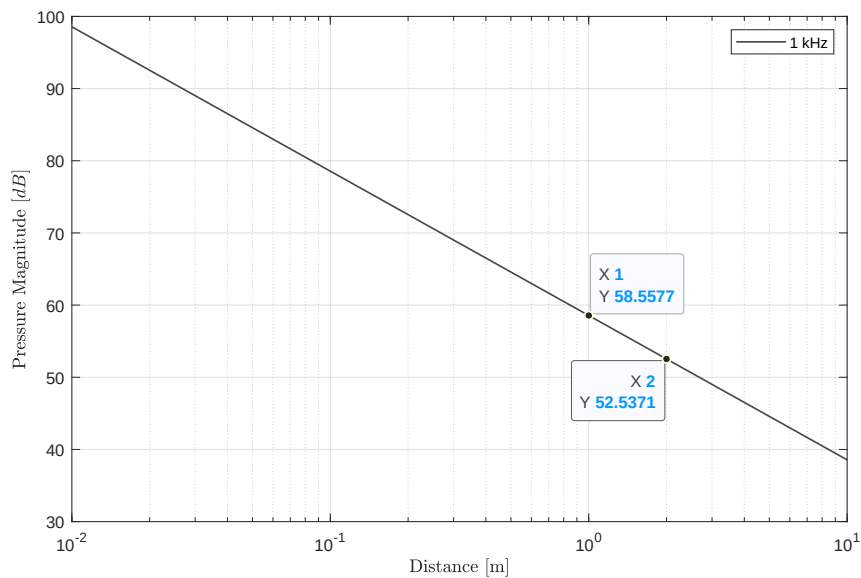


Figure 2: Monopole pressure drop versus distance for 1 kHz.

Doubling the distance yields a -6 dB of pressure level change (see data points).

Problem 1b

Figure 3 illustrates the directivity patterns for a monopole, dipole, lateral quadrupole, and a linear quadrupole for 1 kHz. The receivers were placed at a 1 m distance around each of the sources.

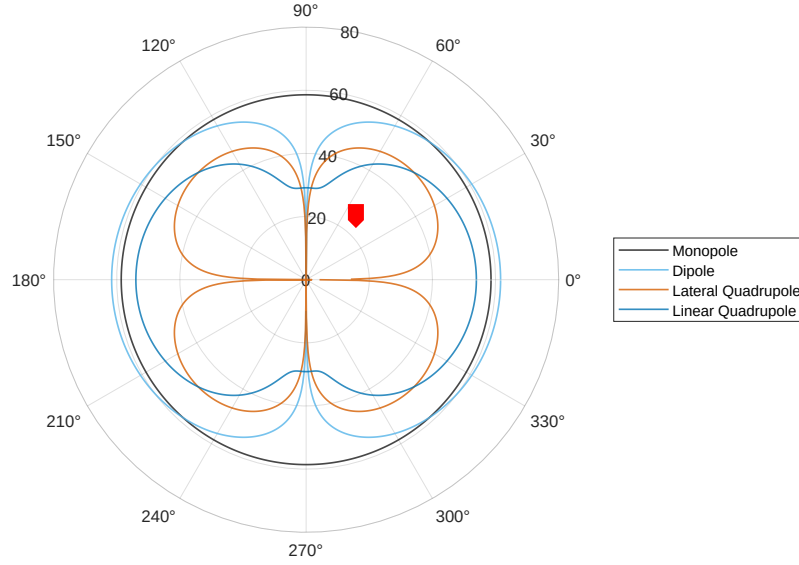


Figure 3: Directivity patterns for a 1 kHz monopole, dipole, lateral quadrupole, and linear quadrupole.

Table 1 lists the sound pressure levels for the 4 types of patterns for 1 kHz.

Source Type	Sound Pressure Level [dB]
Monopole	58.6
Dipole	61.6
Lateral Quadrupole	48.2
Linear Quadrupole	53.9

Table 1: Sound pressure levels for 1 kHz at 1 m distance.

The monopole had a volume velocity of $1.4 \frac{\text{m}^3}{\text{s}}$.

The dipole had two equal strength, in-phase elements with a separation of 0.0429 m, which is less than the 1 kHz wavelength of 0.343 m. The volume velocity for both sources was $1.4 \frac{\text{m}^3}{\text{s}}$.

The lateral quadrupole had four equal strength, diagonally opposite in-phase pairs with a separation of 0.0429 m. The volume velocity for both sources was $1.4 \frac{\text{m}^3}{\text{s}}$.

The linear quadrupole had three sources, the end sources were equal strength and in-phase, while the center source was twice the strength of the single sources and 180° out-of-phase. The separation between all three sources was 0.0429 m. The volume velocity of the end sources was $1.4 \frac{\text{m}^3}{\text{s}}$ and the volume velocity for the center source was $2.8 \frac{\text{m}^3}{\text{s}}$.

Problem 1c

Figure 4 shows a set of directivity patterns for a 9-element line source with a separated of 0.0429 m centered at the origin along the x-axis (demarcated by the 180° to 0° line in the polar plot) for 100 Hz, 2 kHz, and 5 kHz. The receivers are placed at a 1 m distance around the line source.

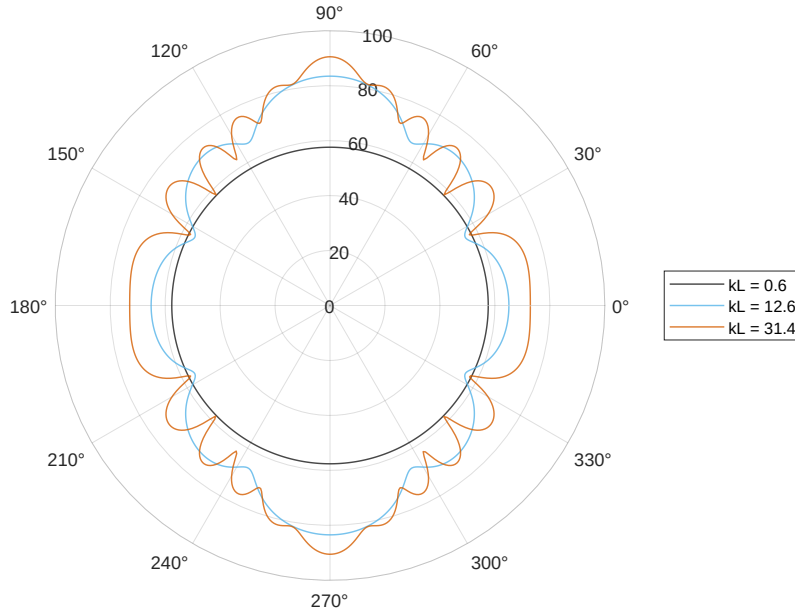


Figure 4: Line source directivity patterns for frequencies of 100 Hz, 2 kHz, and 5 kHz with small source separation.

The length of the array is 0.343 m, or the 1 kHz wavelength. As the frequency of the source increases (i.e., $k \cdot L$), the patterns have larger directivity but with more side lobes.

Figure 5 shows the directivity patterns for a similar line source, but with the sources now separated by 0.172 m.

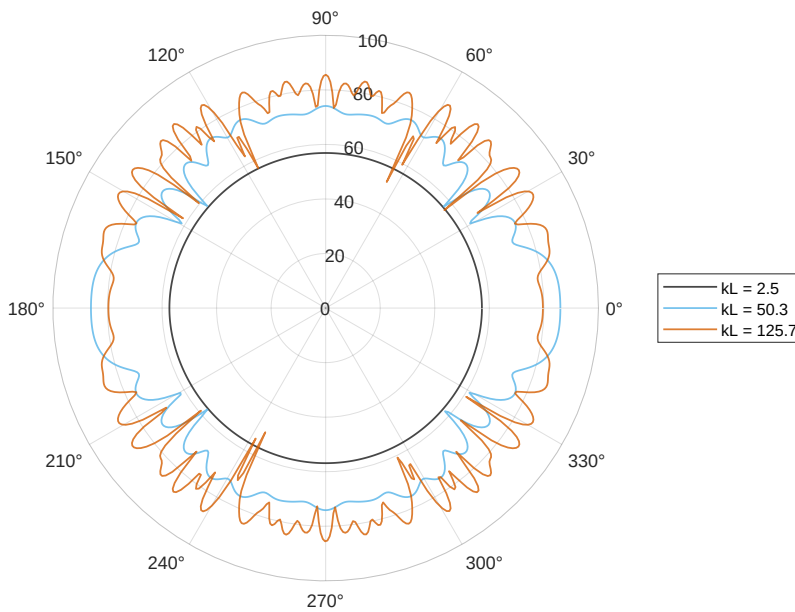


Figure 5: Line source directivity patterns for frequencies of 100 Hz, 2 kHz, and 5 kHz with large source separation.

The length of the array is now 1.37 m, or 4 times the 1 kHz wavelength. As the frequency of the source increases (i.e., $k \cdot L$), the patterns have larger directivity but with more side lobes. Here, unlike the previous case, there are a greater number of side lobes for the 2 kHz signal (blue curve).

Problem 1d

Figure 6 shows the monopole source locations in the xy-plane. The number of sources in the plane was 100 and “a” was set to 1.715 m (5 times the 1 kHz wavelength of 0.343 m).

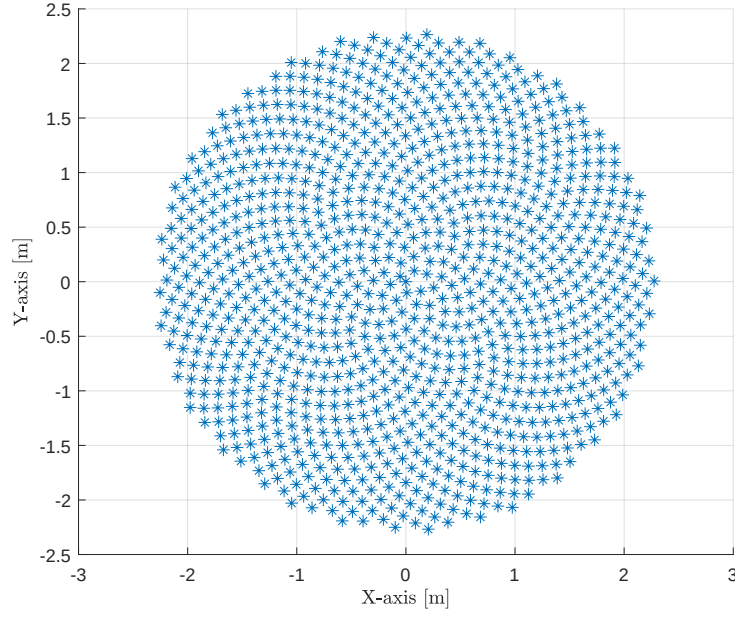


Figure 6: Monopole source locations in the xy-plane.

Figure 7 shows the estimated and theoretical null locations along the z-axis above the xy-plane for the baffled piston.

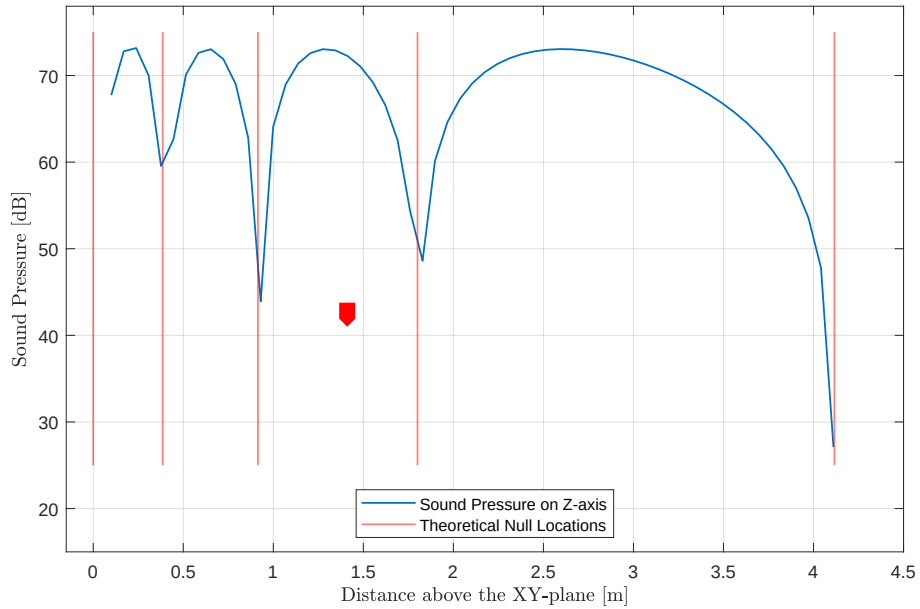


Figure 7: The estimated and theoretical null locations along the z-axis for the baffled piston.

The factor $\frac{a}{\lambda}$ was 5; there are 5 theoretical null locations. There is good agreement between the estimated and theoretical null points. The estimated null near the surface of the piston is governed by starting receiver location on the z-axis. As this distance is made smaller, the sound pressure level grows rapidly due to the calculation method of the monopole sources.

Problem 2 - Active Control of Enclosure Opening

The Matlab code for this problem is listed in Appendix 2.

Problem 2a

Figure 1 shows the sound pressure level reduction versus octave band frequency for the linear quadrupole.

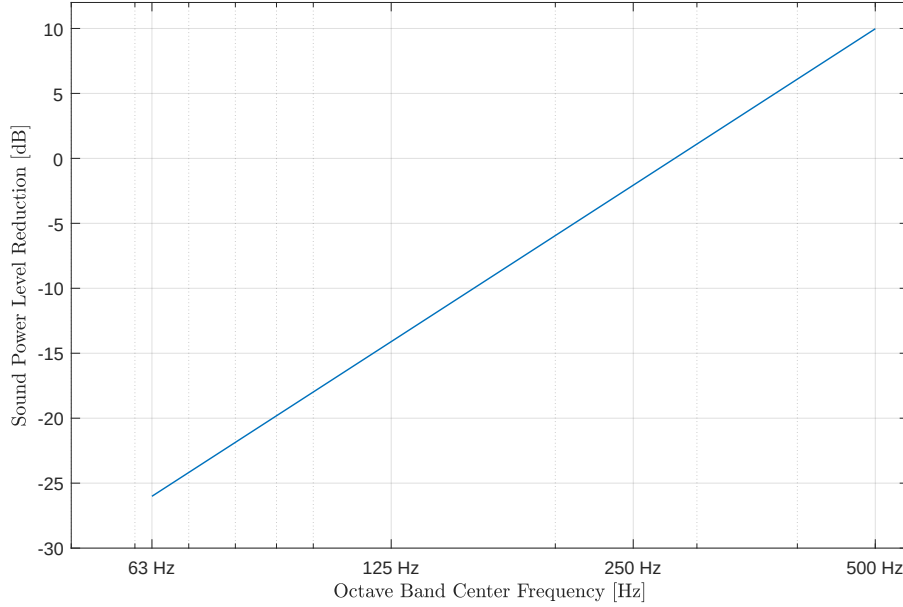


Figure 1: Sound pressure level reduction as a function of octave band center frequency.

Problem 2b

Figure 2 shows the octave sound level reduction versus the angle, θ , between the plane of the opening and a line at the center of the box opening.

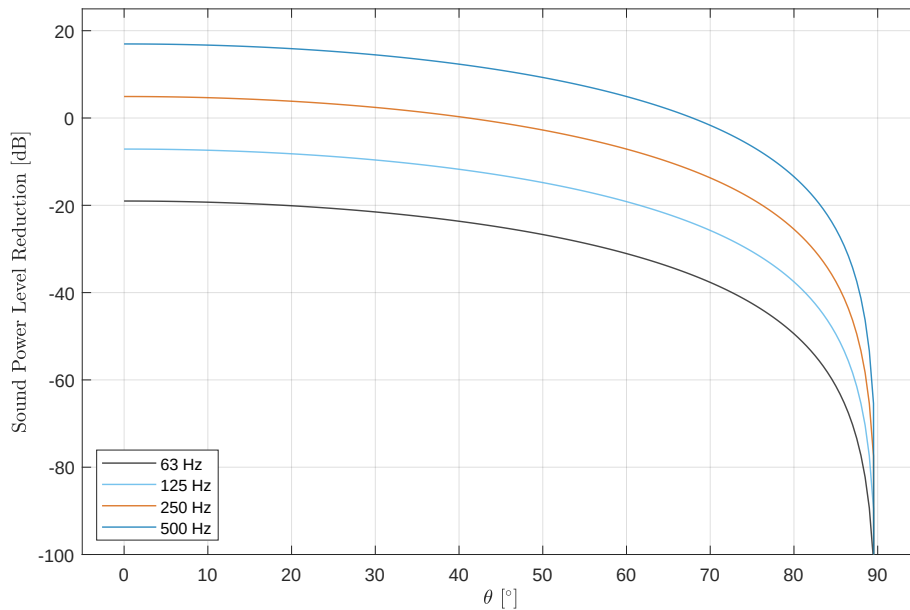


Figure 2: Octave band pressure level reduction as a function theta.

The relative level reduction across the four octave bands is similar. As the receiver moves from the side of the opening towards its center, the level of reduction increases, where at 90° , a null is reached as shown in Figure 3 for Problem 1b.

Problem 3 - Coherent and Incoherent Speakers

The Matlab code for this problem is listed in Appendix 3.

Part 3a

The expected sound pressure at the receive for the coherent sources is -17.3 dB.

Part 3b

The expected sound pressure at the receive for the incoherent sources is -0.71 dB.

Problem 4 - Traffic Noise Modeling

The Matlab code for this problem is listed in Appendix 4.

Part 4a

If the line of traffic is infinitely long, the estimated sound pressure level at the house is approximately 100 dB.

From slide 27 of Lecture 22 (Monday, April 2, 2025), the first knee-point is $\frac{D}{3}$ is 2.67 m, while the second knee-point is infinite because of the assumption of an infinitely long line of traffic. This requires that the decay per doubling of distance is -3 dB.

Figure 1 shows the estimated sound pressure level versus number of line sources.

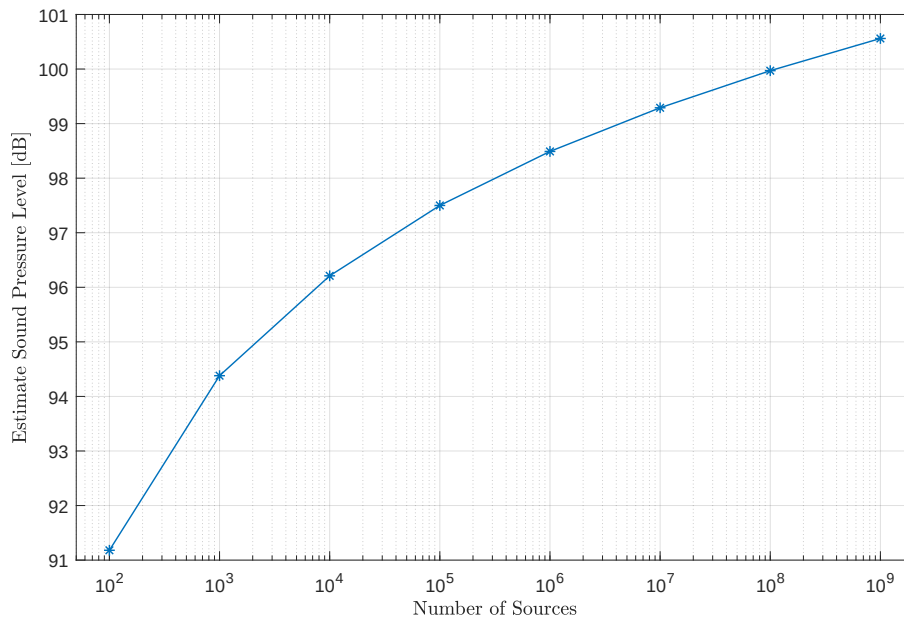


Figure 1: Estimated sound pressure level versus number of line sources.

Part 4b

When the line source circles the house at a 200 m radius, there will be a finite number of incoherent sources around the house. The number of estimate sources is 157, which produces a pressure level at the house of about 93 dB.

1 Appendix - Matlab Code for Problem 1

```
1
2
3
4 %% Synopsis
5
6 % Problem 1 – Simulation of Sources from Monopoles
7
8 % See Lecture 20
9 % "D:\15 Downloads\00 GitHub\ACS_547\35 Lectures\20 Monday, March 31, 2025\Lecture 20 – Dipoles and
   quadrupoles – Filled.pptx"
10
11
12
13 %% Note(s)
14
15 % Distance (source and receivers) is in units of meters.
16
17 % A source can be real-valued or complex-valued. Complex-valued sources have magnitude and phase
   information.
18
19 % A sources has units of volume velocity,  $\text{m}^3/\text{s}$ .
20
21 % Complex pressures are in units of Pascals.
22
23
24
25 %% Environment
26
27 close all; clear; clc;
28 % restore default path;
29
30 addpath( genpath( './00 Support' ), '-begin' );
31
32 % set( groot, 'DefaultFigurePosition', [ 1.7e3 775 750 500 ] ); % x, y, width, height
33
34 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
35 set( 0, 'DefaultFigureWindowStyle', 'norma' );
36 set( 0, 'DefaultLineLineWidth', 1.0 );
37 set( 0, 'DefaultTextInterpreter', 'Latex' );
38
39 format ShortG;
40
41 pause( 1 );
42
43
44
45 %% Constants and Parameters
46
47 c = 343; % m/s
48 rho0 = 1.21; % kg/m^3
49
50 frequency = 1e3; % Hz
51 wavelength = c / frequency; % 0.343 m
52
53 k = (2*pi*frequency) ./ c; % 18.3 1/m (alternative calculation: 2pi / wavelength)
54
55
56 r = [ 0.5 2.4 4.8 9.6 ]; % m; multiplier of 1.
57
58 theta = ( 0:0.01:2*pi ).'; % radians
59
60 xyz_receivers = nan( numel( r ), numel( theta ), 3 ); % m
61
62 for radius_index = 1:1:numel( r )
63     x = r( radius_index )*cos( theta ); y = r( radius_index )*sin( theta ); z = zeros( size( x ) );
64     xyz_receivers( radius_index, :, : ) = [ x(:) y(:) z(:) ];
65 end
66
67
68 color_map = slanCM( 'ColorBlind', 4 );
69
70
71
72 %% Problem 2a – Pressure Versus Distance for a Monopole Source
73
74 % Monopole sound pressure verus distance has a decay of 6 dB per doubling of distance.
75
76 xyz_sources = [ ...
77     0, 0, 0; ...
78 ];
79
80 %
81 Q_sources = 1.4; %  $\text{m}^3/\text{s}$ 
82
83 x = 0.01:0.01:10;
84 y = zeros( length( x ), 1 ); z = zeros( length( x ), 1 );
85 xyz_receivers = [ x(:) y(:) z(:) ];
86
87
```

```

88 figure( 'Name', 'Monopole Source – Pressure Magnitude Versus Distance' ); ...
89
90 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, frequency, rho0, c );
91 L = 10*log10( abs(p).^2 );
92 plot( x, L, 'Color', [ color_map( 1, : ) 0.8 ] ); grid on;
93 legend( '1 kHz', 'Location', 'NorthEast' );
94 xlabel( 'Distance [m]' ); ylabel( 'Pressure Magnitude [dB]' );
95 set( gca, 'XScale', 'log' );
96
97
98 % Note(s):
99
100 % With a higher frequency, the wave number, "k" will be larger. Pressure will be higher at the same
101 % distance.
102 % For a doubling in frequency, the pressure will be 20 dB larger.
103
104
105 %% Problem 2b – Directivity Pattern for a Monopole Source
106
107 xyz_sources = [ ...
108     0, 0, 0; ...
109 ]; % m
110
111 %
112 Q_sources = 1.4; % m3/s
113
114 xyz_receivers = [ cos( theta ) sin( theta ) zeros( size( cos( theta ) ) ) ];
115
116 figure( 'Name', 'Monopole Source – Directivity Pattern' ); ...
117
118 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, frequency, rho0, c );
119 monopole.L = 10*log10( abs(p).^2 );
120
121 polarplot( theta, monopole.L, 'Color', [ color_map( 1, : ) 0.2 ] ); hold on;
122 tick_marks = monopole.L( 1 );
123
124 p2 = sum_of_monopoles( xyz_sources, Q_sources, 4.*xyz_receivers, frequency, rho0, c );
125 monopole.L2 = 10*log10( abs(p2).^2 );
126
127 polarplot( theta, monopole.L2, 'Color', [ color_map( 1, : ) 0.8 ] ); hold on;
128 tick_marks2 = monopole.L2( 1 );
129
130 monopole.label = sprintf( '1 kHz, Q = 1.4 %s @ %1g m, %3.1f dB', '$\frac{m^3}{s}$', string( [ 1
131     round( tick_marks, 1 ) ] ) );
132 monopole.label2 = sprintf( '1 kHz, Q = 1.4 %s @ %1g m, %3.1f dB', '$\frac{m^3}{s}$', string( [ 4
133     round( tick_marks2, 1 ) ] ) );
134
135 rlim( [ 0 80 ] );
136 rticklabels( ...
137     { sprintf( '1 kHz, @ %1g m, %3.1f dB', string( [ 1 round( tick_marks, 1 ) ] ) ), ...
138       sprintf( '1 kHz, @ %1g m, %3.1f dB', string( [ 4 round( tick_marks2, 1 ) ] ) ) } );
139 rtickangle( 0 );
140 rticks( [ tick_marks2 tick_marks ] );
141
142 % figure( 'Name', 'Monopole Source – Pressure Versus Angle' ); ...
143 %
144 % plot( theta.*180./pi, monopole.L, 'Color', [ color_map( 1, : ) 0.8 ] ); grid on;
145 % legend( monopole.label, 'Location', 'North', 'Interpreter', 'Latex' );
146 % xlabel( 'Angle [Degree]' ); ylabel( 'Pressure at 1 m [dB]' );
147 % axis( [ -10 370 0 80 ] );
148
149
150 %% Problem 2b – Directivity Pattern for a Dipole Source
151
152 d = wavelength / 8; % 0.043 m; less than the wavelength of 0.343 m.
153 k*d; % 0.79 unitless
154
155 xyz_sources = [ ...
156     -d, 0, 0; ...
157     +d, 0, 0; ...
158 ]; % m
159
160 %
161 Q_sources = [ ...
162     +1.4; ...
163     -1.4; ...
164 ]; % m3/s
165
166 xyz_receivers = [ cos( theta ) sin( theta ) zeros( size( cos( theta ) ) ) ];
167
168 figure( 'Name', 'Dipole Source – Directivity Pattern' ); ...
169
170 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, frequency, rho0, c );
171 dipole.L = 10*log10( abs(p).^2 );
172
173 polarplot( theta, dipole.L, 'Color', [ color_map( 2, : ) 0.8 ] );
174 tick_marks = max( dipole.L );
175

```

```

176     dipole.label = sprintf( '1 kHz, Q = 1.4 %s @ %lg m, %3.1f dB', '$\frac{m^3}{s}$', string( [ 1
177         round( tick_marks, 1 ) ] ) );
178
179     rlim( [ 0 80 ] );
180     rticklabels( sprintf( '1 kHz, @ %lg m, %3.1f dB', string( [ 1 round( tick_marks, 1 ) ] ) ) );
181     rtickangle( 45 );
182     rticks( tick_marks );
183
184 % figure( 'Name', 'Dipole Source - Pressure Versus Angle' ); ...
185 %
186 %     plot( theta.*180./pi, dipole.L, 'Color', [ color_map( 2, : ) 0.8 ] ); grid on;
187 %     legend( dipole.label, 'Location', 'North', 'Interpreter', 'Latex' );
188 %     xlabel( 'Angle [Degree]' ); ylabel( 'Pressure at 1 m [dB]' );
189 %     axis( [ -10 370 0 80 ] );
190
191
192 %% Problem 2b - Lateral Quadrupole Directivity Pattern
193
194 d = ( wavelength / 8 ) / 2; % 0.043 m; less than the wavelength of 0.343 m.
195
196 xyz_sources = [ ...
197     +d, +d, 0; ...
198     -d, +d, 0; ...
199     -d, -d, 0; ...
200     +d, -d, 0; ...
201 ]; % m
202 %
203 Q_sources = [ ...
204     +1.4; ...
205     -1.4; ...
206     +1.4; ...
207     -1.4; ...
208 ]; % m^3/s
209
210
211 xyz_receivers = [ cos( theta ) sin( theta ) zeros( size( cos( theta ) ) ) ];
212
213 figure( 'Name', 'Lateral Quadrupole Source - Directivity Pattern' ); ...
214
215 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, frequency, rho0, c );
216 lateral_quadrupole.L = 10*log10( abs(p).^2 );
217
218 polarplot( theta, lateral_quadrupole.L, 'Color', [ color_map( 3, : ) 0.8 ] );
219 tick_marks = max( lateral_quadrupole.L );
220
221 lateral_quadrupole.label = sprintf( '1 kHz, Q = 1.4 %s @ %lg m, %3.1f dB', '$\frac{m^3}{s}$',
222     string( [ 1 round( tick_marks, 1 ) ] ) );
223
224 rlim( [ 0 80 ] );
225 rticklabels( sprintf( '1 kHz, @ %lg m, %3.1f dB', string( [ 1 round( tick_marks, 1 ) ] ) ) );
226 rtickangle( 45 );
227 rticks( tick_marks );
228
229 % figure( 'Name', 'Lateral Quadrupole Source - Pressure Versus Angle' ); ...
230 %
231 %     plot( theta.*180./pi, lateral_quadrupole.L, 'Color', [ color_map( 3, : ) 0.8 ] ); grid on;
232 %     legend( lateral_quadrupole.label, 'Location', 'North', 'Interpreter', 'Latex' );
233 %     xlabel( 'Angle [Degree]' ); ylabel( 'Pressure at 1 m [dB]' );
234 %     axis( [ -10 370 0 100 ] );
235
236
237 %% Problem 2b - Linear Quadrupole Directivity Pattern
238
239 d = wavelength / 8; % m
240
241 xyz_sources = [ ...
242     +d, 0, 0; ...
243     0, 0, 0; ...
244     -d, 0, 0; ...
245 ]; % m
246 %
247 Q_sources = [ ...
248     -1.4; ...
249     +2.8; ...
250     -1.4; ...
251 ]; % m^3/s
252
253
254 xyz_receivers = [ cos( theta ) sin( theta ) zeros( size( cos( theta ) ) ) ];
255
256 figure( 'Name', 'Linear Quadrupole Source - Directivity Pattern' ); ...
257
258 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, frequency, rho0, c );
259 linear_quadrupole.L = 10*log10( abs(p).^2 );
260
261 polarplot( theta, linear_quadrupole.L, 'Color', [ color_map( 4, : ) 0.8 ] );
262 tick_marks = max( linear_quadrupole.L );
263

```

```

264 linear_quadrupole.label = sprintf( '1 kHz, Q = 1.4 %s @ %lg m, %3.1f dB', '$\frac{m^3}{s}$', string
    ( [ 1 round( tick_marks, 1 ) ] ) );
265
266 rlim( [ 0 80 ] );
267 rticklabels( sprintf( '1 kHz, @ %lg m, %3.1f dB', string( [ 1 round( tick_marks, 1 ) ] ) ) );
    rtickangle( 45 );
268 rticks( tick_marks );
269
270
271 % figure( 'Name', 'Linear Quadrupole Source — Pressure Versus Angle' ); ...
272 %
273 %     plot( theta.*180./pi, linear_quadrupole.L, 'Color', [ color_map( 4, : ) 0.8 ] ); grid on;
274 %     legend( linear_quadrupole.label, 'Location', 'North', 'Interpreter', 'Latex' );
275 %     xlabel( 'Angle [Degree]' ); ylabel( 'Pressure at 1 m [dB]' );
276 %     axis( [ -10 370 0 100 ] );
277
278
279
280 %% Problem 2b — Combined Plot
281
282 figure( 'Name', 'Combined Directivity Patterns — Polar Plot' ); ...
283
284 h_1 = polarplot( theta, monopole.L, 'Color', [ color_map( 1, : ) 0.8 ] ); hold on;
285 h_2 = polarplot( theta, dipole.L, 'Color', [ color_map( 2, : ) 0.8 ] );
286 h_3 = polarplot( theta, lateral_quadrupole.L, 'Color', [ color_map( 3, : ) 0.8 ] );
287 h_4 = polarplot( theta, linear_quadrupole.L, 'Color', [ color_map( 4, : ) 0.8 ] );
288 legend( [ h_1, h_2, h_3, h_4 ], { 'Monopole', 'Dipole', 'Lateral Quadrupole', 'Linear
    Quadrupole' }, 'Location', 'EastOutside' );
289
290 %
291 rlim( [ 0 80 ] );
292
293 % figure( 'Name', 'Combined Directivity Patterns — Pressure Versus Angle' ); ...
294 %
295 %     h_1 = plot( theta.*180./pi, monopole.L, 'Color', [ color_map( 1, : ) 0.8 ] ); hold on;
296 %     h_2 = plot( theta.*180./pi, dipole.L, 'Color', [ color_map( 2, : ) 0.8 ] );
297 %     h_3 = plot( theta.*180./pi, lateral_quadrupole.L, 'Color', [ color_map( 3, : ) 0.8 ] );
298 %     h_4 = plot( theta.*180./pi, linear_quadrupole.L, 'Color', [ color_map( 4, : ) 0.8 ] ); grid on;
299 %     legend( linear_quadrupole.label, 'Location', 'North', 'Interpreter', 'Latex' );
300 %     xlabel( 'Angle [Degree]' ); ylabel( 'Pressure at 1 m [dB]' );
301 %     axis( [ -10 370 0 90 ] );
302
303
304
305 %% Problem 2c — Finite Line Source — 4 Sources Uniformly Separation — Less than a Wavelength
306
307 % See Lecture 21 — Distributed Sources (D:\15 Downloads\00 GitHub\ACS_547\35 Lectures\21 Wednesday,
    April 2, 2025)
308
309
310 d = wavelength / 8; % Source separation distance.
311
312 xyz_sources = [ ...
313     -4*d, 0, 0; ...
314     -3*d, 0, 0; ...
315     -2*d, 0, 0; ...
316     -1*d, 0, 0; ...
317     0, 0, 0; ...
318     1*d, 0, 0; ...
319     2*d, 0, 0; ...
320     3*d, 0, 0; ...
321     4*d, 0, 0; ...
322 ]; % m
323
324 Q_sources = [ ...
325     +1.4 + 1i*0; ...
326     +1.4 + 1i*0; ...
327     +1.4 + 1i*0; ...
328     +1.4 + 1i*0; ...
329     +1.4 + 1i*0; ...
330     +1.4 + 1i*0; ...
331     +1.4 + 1i*0; ...
332     +1.4 + 1i*0; ...
333     +1.4 + 1i*0; ...
334 ]; % m^3/s
335
336
337 xyz_receivers = [ cos( theta ) sin( theta ) zeros( size( cos( theta ) ) ) ];
338
339 array_length = ( size( xyz_sources, 1 ) - 1 )*d; % 0.343 m
340
341
342 figure( 'Name', 'Finite Line Source — 4 Sources Uniformly Spaced' ); ...
343
344 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, 1e2, rho0, c );
345 L = 10*log10( abs(p).^2 );
346 polarplot( theta, L, 'Color', [ color_map( 1, : ) 0.8 ] ); hold on;
347
348 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, 2e3, rho0, c );
349 L = 10*log10( abs(p).^2 );
350 polarplot( theta, L, 'Color', [ color_map( 2, : ) 0.8 ] );
351

```

```

352 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, 5e3, rho0, c );
353 L = 10*log10( abs(p).^2 );
354 polarplot( theta, L, 'Color', [ color_map( 3, : ) 0.8 ] ); grid on;
355
356 legend( sprintf( 'kL = %3.1f', (2*pi*1e2)/c*array_length ), sprintf( 'kL = %3.1f', (2*pi*2e3)/c*
    array_length ), sprintf( 'kL = %3.1f', (2*pi*5e3)/c*array_length ), 'Location', 'EastOutside' )
    ;
357 rlim( [ 0 1e2 ] );
358
359
360
361 %% Problem 2c – Finite Line Source – 4 Sources Uniformly Separation – 1 kHz Wavelength Separation
362
363 % See Lecture 21 – Distributed Sources (D:\15 Downloads\00 GitHub\ACS_547\35 Lectures\21 Wednesday,
    April 2, 2025)
364
365
366 d = wavelength / 2;
367
368 xyz_sources = [ ...
369     -4*d, 0, 0; ...
370     -3*d, 0, 0; ...
371     -2*d, 0, 0; ...
372     -1*d, 0, 0; ...
373     0, 0, 0; ...
374     1*d, 0, 0; ...
375     2*d, 0, 0; ...
376     3*d, 0, 0; ...
377     4*d, 0, 0; ...
378 ]; % m
379
380 Q_sources = [ ...
381     +1.4 + 1i*0; ...
382     +1.4 + 1i*0; ...
383     +1.4 + 1i*0; ...
384     +1.4 + 1i*0; ...
385     +1.4 + 1i*0; ...
386     +1.4 + 1i*0; ...
387     +1.4 + 1i*0; ...
388     +1.4 + 1i*0; ...
389     +1.4 + 1i*0; ...
390 ]; % m^3/s
391
392
393 xyz_receivers = [ cos( theta ) sin( theta ) zeros( size( cos( theta ) ) ) ];
394
395 array_length = ( size( xyz_sources, 1 ) - 1 )*d; % 1.37 m
396
397
398 figure( 'Name', 'Finite Line Source – 4 Sources Uniformly Spaced' ); ...
399
400 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, 1e2, rho0, c );
401 L = 10*log10( abs(p).^2 );
402 polarplot( theta, L, 'Color', [ color_map( 1, : ) 0.8 ] ); hold on;
403
404 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, 2e3, rho0, c );
405 L = 10*log10( abs(p).^2 );
406 polarplot( theta, L, 'Color', [ color_map( 2, : ) 0.8 ] );
407
408 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, 5e3, rho0, c );
409 L = 10*log10( abs(p).^2 );
410 polarplot( theta, L, 'Color', [ color_map( 3, : ) 0.8 ] ); grid on;
411
412 legend( sprintf( 'kL = %3.1f', (2*pi*1e2)/c*array_length ), sprintf( 'kL = %3.1f', (2*pi*2e3)/c*
    array_length ), sprintf( 'kL = %3.1f', (2*pi*5e3)/c*array_length ), 'Location', 'EastOutside' )
    ;
413 rlim( [ 0 1e2 ] );
414
415
416
417 %% Problem 2d – Baffled Piston
418
419 % See Lecture 21 – Distributed Sources (D:\15 Downloads\00 GitHub\ACS_547\35 Lectures\21 Wednesday,
    April 2, 2025)
420
421 % See: https://demonstrations.wolfram.com/SunflowerSeedArrangements/
422
423
424 a = 5*wavelength;
425
426
427 N = 1e2;
428 n = 0:1:( N - 1 );
429
430 phi = ( 1 + sqrt(5) ) / 2;
431
432 theta_n = (2*pi)/(phi^2) .* n;
433 r_n = a.*( sqrt( n./N ) );
434
435 x_n = r_n.*cos(theta_n); y_n = r_n.*sin(theta_n);
436 xyz_sources = [ x_n(:) y_n(:) zeros( N, 1 ) ];
437 %

```

```

438 % figure( 'Name', 'Distribution of Sources on Baffled Piston' ); ...
439 % scatter( x_n, y_n, 40.*ones( size(x_n) ), 'Marker', '*' ); grid on;
440 % xlabel( 'X-axis [m]' ); ylabel( 'Y-axis [m]' );
441 % daspect( [ 1 1 1 ] );
442 %
443 Q_sources = ones( size( xyz_sources, 1 ), 1 );
444
445 z_start = a/10;
446 z_receivers = linspace( 0.1, 20.25*wavelength, 1e2 );
447 z_receivers( 60:end ) = [ ];
448 xyz_receivers = [ zeros( numel( z_receivers ), 1 ) zeros( numel( z_receivers ), 1 )
449                   z_receivers(:) ];
450
451 m = 1:1:( a / wavelength );
452 r_null = ( (a/wavelength)^2 - m.^2 ) ./ (2*m/wavelength);
453
454 figure( 'Name', 'Baffled Piston - ka = 8' ); ...
455
456 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receivers, frequency, rho0, c );
457 L = 10*log10( abs(p).^2 );
458
459 h1 = plot( z_receivers, L ); hold on;
460 h2 = line( [ r_null(1) r_null(1) ], [ 25 75 ], 'Color', [ 1, 0, 0, 0.5 ] );
461 line( [ r_null(2) r_null(2) ], [ 25 75 ], 'Color', [ 1, 0, 0, 0.5 ] );
462 line( [ r_null(3) r_null(3) ], [ 25 75 ], 'Color', [ 1, 0, 0, 0.5 ] );
463 line( [ r_null(4) r_null(4) ], [ 25 75 ], 'Color', [ 1, 0, 0, 0.5 ] );
464 line( [ r_null(5) r_null(5) ], [ 25 75 ], 'Color', [ 1, 0, 0, 0.5 ] ); grid on;
465 legend( [ h1 h2 ], { 'Sound Pressure on Z-axis', 'Theoretical Null Locations' }, 'Location', '
466         South' );
467 axis( [ -0.15 4.5 15 78 ] );
468 xlabel( 'Distance above the XY-plane [m]' ); ylabel( 'Sound Pressure [dB]' );
469
470
471 %% Clean-up
472
473 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
474     monitors = get( 0, 'MonitorPositions' );
475     if ( size( monitors, 1 ) == 1 )
476         autoArrangeFigures( 3, 4, 1 );
477     elseif ( 1 < size( monitors, 1 ) )
478         autoArrangeFigures( 2, 2, 2 );
479     end
480 end
481
482 fprintf( 1, '\n\n*** Processing Complete ***\n\n\n' );
483
484
485
486 %% Reference(s)
487
488 p = 10 + 1i*5; % Pascals; sinusoid
489
490 p_mag = abs( p ); % 11.18 Pascals
491
492 p_rms = p_mag / sqrt(2); % 7.91 Pascals RMS
493
494 p_dB_SPL = 20*log10( p_rms / 20e-6 ); % 111.94 dB SPL Z
495
496
497 p_dB_SPL_verify = convert_complex_pressure_to_dB_SPL( p );
498
499
500
501
502 %% URLs
503
504 % https://www.mathworks.com/matlabcentral/answers/1771015-how-to-customize-a-colormap
505
506 % https://www.mathworks.com/help/matlab/ref/colororder.html#mw\_a3ac0dbb-c8d8-48c9-977c-1bfa84840b08
507
508 % https://www.mathworks.com/matlabcentral/answers/493751-increase-levels-on-colorbar
509
510 % https://www.mathworks.com/matlabcentral/answers/216283-get-current-axes-from-multiple-figures
511
512 % https://www.mathworks.com/matlabcentral/answers/288557-how-to-join-every-row-of-an-array-of-strings-in-a-single-string-for-time-plotting-purposes
513
514 % https://www.mathworks.com/matlabcentral/answers/352290-set-colorbar-ticklabels-and-tickmarks
515
516 % https://www.mathworks.com/matlabcentral/answers/152426-sprintf-d-x-prints-out-exponential-notation-instead-of-decimal-notation

```

2 Appendix - Matlab Code for Problem 2

```
1
2
3
4 %% Synopsis
5
6 % Problem 2 — Active Control of Enclosure Opening
7
8 % See Lecture 22
9 % "D:\15 Downloads\00 GitHub\ACS_547\35 Lectures\22 Monday, April 7, 2025\Lecture 22 — Coherence
   effects — filled.pptx"
10
11
12
13 %% Note(s)
14
15 % Distance (source and receivers) are in units of meters.
16
17 % Source can be real-valued or complex-valued. Complex-valued sources have magnitude and phase
   information.
18
19 % Sources have units of volume velocity,  $\text{m}^3/\text{s}$ .
20
21 % Complex pressures are in units of Pascals.
22
23
24
25 %% Environment
26
27 close all; clear; clc;
28 % restore default path;
29
30 addpath( genpath( './00 Support' ), '-begin' );
31
32 set( groot, 'DefaultFigurePosition', [ 1.7e3 775 750 500 ] ); % x, y, width, height
33
34 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
35 set( 0, 'DefaultFigureWindowStyle', 'normal' );
36 set( 0, 'DefaultLineLineWidth', 0.8 );
37 set( 0, 'DefaultTextInterpreter', 'Latex' );
38
39 format ShortG;
40
41 pause( 1 );
42
43
44
45 %% Define Anonymous Function(s)
46
47 h_sound_power_ratio = @( k, d ) ( 4/5 ) .* ( k .* d ).^4;
48
49 h_pressure_ratio = @( theta, sound_level_ratio ) 5 .* cos( theta ).^4 * sound_level_ratio;
50
51
52
53 %% Constants and Parameters
54
55 c = 343; % m/s
56 rho0 = 1.21; % kg/m^3
57
58 frequencies = [ 63 125 250 500 ]; % Hz
59 wavelengths = c ./ frequencies; % 5.44 m, 2.74 m, 1.37 m, and 0.69 m
60
61 k = (2*pi*frequencies) ./ c; % 1.15 1/m, 2.29 1/m, 4.58 1/m, and 9.16 1/m
62
63
64 color_map = slanCM( 'ColorBlind', 4 );
65
66
67
68 %% Problem 2a
69
70 d = 205e-3; % m; distance between each of the two speakers and the center of the square
71
72
73 figure( 'Name', 'Sound Power Level Reductions' ); ...
74
75 semilogx( frequencies, 10*log10( h_sound_power_ratio( k, d ) ) ); grid on;
76 xlabel( 'Octave Band Center Frequency [Hz]' ); ylabel( 'Sound Power Level Reduction [dB]' );
77 xticks( [ 63, 125, 250, 500 ] ); xticklabels( { '63 Hz', '125 Hz', '250 Hz', '500 Hz' } );
78 axis( [ 50 550 -30 12 ] );
79
80
81
82 %% Problem 2b
83
84 theta_degrees = 0:0.5:90;
85 theta_radians = theta_degrees.*pi./180;
86
87
```

```

88 figure( 'Name', 'Sound Power Level Reductions Versus Theta' ); ...
89
90 plot( theta_degrees, 10*log10( h_pressure_ratio( theta_radians, h_sound_power_ratio( k(1), d ) ) ),
91       'Color', [ color_map( 1, : ) 0.8 ] ); hold on;
92 plot( theta_degrees, 10*log10( h_pressure_ratio( theta_radians, h_sound_power_ratio( k(2), d ) ) ),
93       'Color', [ color_map( 2, : ) 0.8 ] );
94 plot( theta_degrees, 10*log10( h_pressure_ratio( theta_radians, h_sound_power_ratio( k(3), d ) ) ),
95       'Color', [ color_map( 3, : ) 0.8 ] );
96 plot( theta_degrees, 10*log10( h_pressure_ratio( theta_radians, h_sound_power_ratio( k(4), d ) ) ),
97       'Color', [ color_map( 4, : ) 0.8 ] ); grid on;
98 legend( '63 Hz', '125 Hz', '250 Hz', '500 Hz', 'Location', 'SouthWest' )
99 xlabel( '$\theta$ [$^\circ]$' ); ylabel( 'Sound Power Level Reduction [dB]' );
100 axis( [ -5 95 -1e2 25 ] );
101
102 %% Clean-up
103 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
104     monitors = get( 0, 'MonitorPositions' );
105     if ( size( monitors, 1 ) == 1 )
106         autoArrangeFigures( 3, 4, 1 );
107     elseif ( 1 < size( monitors, 1 ) )
108         autoArrangeFigures( 2, 2, 2 );
109     end
110 end
111 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
112
113
114
115 %% Reference(s)
116
117 p = 10 + 1i*5; % Pascals; sinusoid
118
119 p_mag = abs( p ); % 11.18 Pascals
120
121 p_rms = p_mag / sqrt(2); % 7.91 Pascals RMS
122
123 p_dB_SPL = 20*log10( p_rms / 20e-6 ); % 111.94 dB SPL Z
124
125
126 p_dB_SPL_verify = convert_complex_pressure_to_dB_SPL( p );
127
128
129
130
131 %% URLs

```

3 Appendix - Matlab Code for Problem 3

```
1
2
3
4 %% Synopsis
5
6 % Problem 3 — Coherent and Incoherent Speakers
7
8 % See Lecture 22 on Monday, April 7, 2025
9 % "D:\15 Downloads\00 GitHub\ACS_547\35 Lectures\22 Monday, April 7, 2025\Lecture 22 — Coherence
   effects — filled.pptx""
10
11
12
13 %% Note(s)
14
15 % Distance (source and receivers) are in units of meters.
16
17 % Source can be real-valued or complex-valued. Complex-valued sources have magnitude and phase
   information.
18
19 % Sources have units of volume velocity,  $\text{m}^3/\text{s}$ .
20
21 % Complex pressures are in units of Pascals.
22
23
24
25 %% Environment
26
27 close all; clear; clc;
28 % restore default path;
29
30 addpath( genpath( './00 Support' ), '-begin' );
31
32 set( groot, 'DefaultFigurePosition', [ 1.7e3 775 750 500 ] ); % x, y, width, height
33
34 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
35 set( 0, 'DefaultFigureWindowStyle', 'norma' );
36 set( 0, 'DefaultLineLineWidth', 0.8 );
37 set( 0, 'DefaultTextInterpreter', 'Latex' );
38
39 format ShortG;
40
41 pause( 1 );
42
43
44
45 %% Define Anonymous Function(s)
46
47 h_volume_velocity = @( rho, c, k, watts ) sqrt( ( watts*8*pi ) / ( rho*c*k^2 ) );
48
49
50
51 %% Constants and Parameters
52
53 c = 343; % m/s
54 rho0 = 1.21; % kg/m^3
55
56 frequency = 125; % Hz
57 wavelengths = c ./ frequency; % 2.74 m
58
59 k = (2*pi*frequency) ./ c; % 2.29 1/m
60
61
62
63 %% Problem 2a
64
65 d = 1e-1; % m
66
67
68 % Calculate volume velocity (see Lecture 19, slide 35).
69 volume_velocity_0_5_watts = h_volume_velocity ( rho0, c, k, 0.5 );
70 volume_velocity_1_0_watts = h_volume_velocity ( rho0, c, k, 1 );
71
72
73 xyz_sources = [ ...
74     +d, 0, 0; ...
75     0, 0, 0; ...
76     -d, 0, 0; ...
77 ]; % m
78
79 Q_sources = [ ...
80     +volume_velocity_0_5_watts; ...
81     -volume_velocity_1_0_watts; ...
82     +volume_velocity_0_5_watts; ...
83 ]; %  $\text{m}^3/\text{s}$ 
84
85 xyz_receiver = [ 20 tand(30)*20 1 ];
86 % xyz_receiver = [ 0 0 1 ];
87
```

```

88 p = sum_of_monopoles( xyz_sources, Q_sources, xyz_receiver, frequency, rho0, c );
89 L = 10*log10( abs(p).^2 ); % -17.3 dB
90
91
92
93 %% Problem 2b
94
95 xyz_sources = [ ...
96     +d, 0, 0; ...
97     0, 0, 0; ...
98     -d, 0, 0; ...
99 ]; % m
100
101 %
102 Q_sources = [ ...
103     +volume_velocity_0_5_watts; ...
104     -volume_velocity_1_0_watts; ...
105     +volume_velocity_0_5_watts; ...
106 ]; % m^3/s
107
108 % Speaker 1
109 p1 = sum_of_monopoles( xyz_sources(1, :), Q_sources(1), xyz_receiver, frequency, rho0, c );
110 L1 = 10*log10( abs( p1 ) ); % -6.0 dB
111
112 % Speaker 2
113 p2 = sum_of_monopoles( xyz_sources(2, :), Q_sources(2), xyz_receiver, frequency, rho0, c );
114 L2 = 10*log10( abs( p2 ) ); % -4.5 dB
115
116 % Speaker 3
117 p3 = sum_of_monopoles( xyz_sources(3, :), Q_sources(3), xyz_receiver, frequency, rho0, c );
118 L3 = 10*log10( abs( p3 ) ); % -6.1 dB
119
120
121 L_incoherent_sources = 10*log10( 10^(L1/10) + 10^(L2/10) + 10^(L3/10) ); % -0.71 dB
122
123
124
125 %% Clean-up
126
127 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
128     monitors = get( 0, 'MonitorPositions' );
129     if ( size( monitors, 1 ) == 1 )
130         autoArrangeFigures( 3, 4, 1 );
131     elseif ( 1 < size( monitors, 1 ) )
132         autoArrangeFigures( 2, 2, 2 );
133     end
134 end
135
136 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
137
138
139
140 %% Reference(s)
141
142 p = 10 + 1i*5; % Pascals; sinusoid
143
144 p_mag = abs( p ); % 11.18 Pascals
145
146 p_rms = p_mag / sqrt(2); % 7.91 Pascals RMS
147
148 p_dB_SPL = 20*log10( p_rms / 20e-6 ); % 111.94 dB SPL Z
149
150
151 p_dB_SPL_verify = convert_complex_pressure_to_dB_SPL( p );
152
153
154
155
156 %% URLs

```

4 Appendix - Matlab Code for Problem 4

```
1
2
3
4 %% Synopsis
5
6 % Problem 4 – Traffic Noise Modeling
7
8 % See Lecture 22 on Monday, April 7, 2025
9 % "D:\15 Downloads\00 GitHub\ACS_547\35 Lectures\22 Monday, April 7, 2025\Lecture 22 – Coherence
   effects – filled.pptx"
10
11
12
13 %% Note(s)
14
15 % Incoherent line source.
16
17 % Distance (source and receivers) are in units of meters.
18
19 % Source can be real-valued or complex-valued. Complex-valued sources have magnitude and phase
   information.
20
21 % Sources have units of volume velocity,  $\text{m}^3/\text{s}$ .
22
23 % Complex pressures are in units of Pascals.
24
25
26
27 %% Environment
28
29 close all; clear; clc;
30 % restoredefaultpath;
31
32 addpath( genpath( './00 Support' ), '-begin' );
33
34 % set( groot, 'DefaultFigurePosition', [ 100 750 750 500 ] ); % x, y, width, height
35
36 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
37 set( 0, 'DefaultFigureWindowStyle', 'normal' );
38 set( 0, 'DefaultLineLineWidth', 0.8 );
39 set( 0, 'DefaultTextInterpreter', 'Latex' );
40
41 format ShortG;
42
43 pause( 1 );
44
45
46
47 %% Define Anonymous Function(s)
48
49 h_distance_to_house = @( horizontal_distance, distance_to_house_from_line_source ) sqrt(
   horizontal_distance^2 + distance_to_house_from_line_source^2 );
50
51 h_source_pressure = @( Lw, distance ) Lw - 10*log10( distance );
52
53 h_sound_pressure_net = @( levels ) 10*log10( sum( 10.^(levels./10) ) );
54
55
56
57 %% Contants and Parameters
58
59 source_separation_distance = 8; % m
60
61 average_A_weighted_source_pressure = 86; % dB SPL A at 1 m
62
63 distance_to_house_from_line_source = 200; % m
64
65 D0 = 2;
66
67 Lw = 86 - 10*log10( D0 / (4*pi*1^2) ); % 94 dB
68
69
70
71 %% Part 4a
72
73 % SOURCE_OFFSET = 64; % 75.74 dB
74
75 % SOURCE_OFFSET = 1e0; % 75.74 dB
76 % SOURCE_OFFSET = 1e1; % 84.07 dB
77 % SOURCE_OFFSET = 1e2; % 91.18 dB
78 % SOURCE_OFFSET = 1e3; % 94.38 dB
79 SOURCE_OFFSET = 1e4; % 96.21 dB
80 % SOURCE_OFFSET = 1e5; % 97.50 dB
81 % SOURCE_OFFSET = 1e6; % 98.49 dB
82 % SOURCE_OFFSET = 1e7; % 99.29 dB
83 % SOURCE_OFFSET = 1e8; % 99.97 dB
84 % SOURCE_OFFSET = 1e9; % 100.56 dB
85 x_axis_multiplier = -SOURCE_OFFSET:1:SOURCE_OFFSET;
86
```

```

87
88 level_set = nan( numel( x_axis_multiplier ), 1 );
89
90 fprintf( 1, '\n' );
91
92 for index = 1:1:numel( x_axis_multiplier )
93
94     horizontal_distance = abs( x_axis_multiplier(index)*source_separation_distance );
95
96     working_distance = h_distance_to_house( horizontal_distance, distance_to_house_from_line_source );
97
98     level_set( index ) = h_source_pressure( Lw, working_distance );
99
100     % fprintf( 1, '%d\t%d m\t\tDistance:  %3.1f\t%3.1f\n', x_axis_multiplier(index),
101         horizontal_distance, round( working_distance, 1 ), level_set(index) );
102
103 end
104
105 fprintf( 1, '\n' );
106
107 % h_sound_pressure_net( level_set )
108
109 number_of_sources = [ 1e2  1e3  1e4  1e5  1e6  1e7  1e8  1e9 ];
110 estimated_levels = [ 91.18  94.38  96.21  97.50  98.49  99.29  99.97  100.56 ];
111
112 figure( ); ...
113     semilogx( number_of_sources, estimated_levels, 'Marker', '*' ); grid on;
114     xlabel( 'Number of Sources' ); ylabel( 'Estimate Sound Pressure Level [dB]' );
115     xlim( [ 5e1 2e9 ] );
116
117
118
119 %% Part 4b
120
121 line_source_circumference = 2*pi*distance_to_house_from_line_source;
122
123 number_of_sources_around_the_house = line_source_circumference / 8; % 157
124
125 single_source_pressure = h_source_pressure( Lw, 200 ); % 70.97 dB
126
127 sound_pressure_net_circle = 10*log10( number_of_sources_around_the_house * 10.^(single_source_pressure
128     ./10) ); % 93 dB
129
130
131 %% Clean-up
132
133 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
134     monitors = get( 0, 'MonitorPositions' );
135     if ( size( monitors, 1 ) == 1 )
136         autoArrangeFigures( 3, 4, 1 );
137     elseif ( 1 < size( monitors, 1 ) )
138         autoArrangeFigures( 2, 2, 2 );
139     end
140 end
141
142 fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );
143
144
145
146 %% Reference(s)

```